

Advanced Reference Manual

Manual Integration of FreeRTOS on the STM32U585 Platform

Target Hardware Core: ARM Cortex-M33 (ARMv8-M Architecture)
Target MCU Vendor: STMicroelectronics (STM32U585AI/ZI)
Toolchain & Compiler: STM32CubeIDE Version 1.15+ / GCC Compiler
Port Variant Deployment: Non-TrustZone Monolithic (ARM_CM33_NTZ)

June 21, 2026

Contents

1	Architectural Foundation & Design Rationale	2
1.1	The ARMv8-M Cortex-M33 Architecture Matrix	2
1.2	Why the Non-TrustZone (NTZ) Port Layer is Required	2
1.3	Dynamic Allocation Analysis: Why Heap 4 Wins	2
2	Project Workspace File Structure Directory Isolation	3
3	IDE Compiler Property Path Configurations	4
4	Kernel Tuning Configurations (FreeRTOSConfig.h)	4
5	Hardware Interrupt Mapping & Low-Level Remapping	6
5.1	The Critical Interrupt Trio	6
6	Main Thread Control Loop Implementation	7
7	Troubleshooting & Compilation Architecture Protocols	9
7.1	Resolving Code Cache Out-of-Sync Errors	9
7.2	Common Build Errors Reference Table	9
7.3	The HAL Timebase Conflict Fix	9

1 Architectural Foundation & Design Rationale

The manual integration of a Real-Time Operating System (RTOS) kernel becomes essential when automated code generation tools impose unwanted security abstractions, project structural constraints, or version blocks. To build an efficient firmware architecture on the high-performance **STM32U585**, you must understand how the hardware layer interacts directly with the software components.

1.1 The ARMv8-M Cortex-M33 Architecture Matrix

The STM32U585 microcontroller is built on the ARM Cortex-M33 processing core, which implements the ARMv8-M main baseline architecture. This core features several distinct differences compared to older platforms (such as the Cortex-M4 or M7 ARMv7-M architectures):

- **TrustZone Security Extension:** Splitting the chip into completely isolated Secure and Non-Secure hardware execution zones.
- **Stacked Hardware Exceptions:** Execution contexts utilize dedicated stack pointers (Process Stack Pointer - PSP, and Main Stack Pointer - MSP) combined with new hardware register protection schemes.
- **Floating-Point Unit (FPU):** Dedicated single-precision IEEE 754 hardware floating-point math support requiring explicit save/restore routines during task context switches.

1.2 Why the Non-TrustZone (NTZ) Port Layer is Required

By selecting the `ARM_CM33_NTZ` port layer directory over standard variants, you are enforcing a **monolithic firmware execution framework**.

Critical Engineering Rationale

If your project does not explicitly initialize separate secure and non-secure application binaries via custom bootloaders, a standard TrustZone-enabled RTOS kernel port will fault immediately. The NTZ port bypasses complex secure gateway calls (SG instructions) and directly interfaces with the primary core registers, preventing illegal memory access faults while significantly reducing CPU cycle overhead.

1.3 Dynamic Allocation Analysis: Why Heap 4 Wins

FreeRTOS offers five distinct memory management models (`heap_1.c` through `heap_5.c`). For robust, production-grade applications on the STM32U585, **Heap 4** serves as the optimal choice.

Heap Variant	Memory Fragmentation Profile	Operational Suitability for STM32U5
<code>heap_1.c</code>	Zero Fragmentation (Allocation Only)	Disallowed: Tasks/Queues cannot be deleted.
<code>heap_2.c</code>	High Fragmentation Risk	Disallowed: Lacks adjacent memory block merging.
<code>heap_3.c</code>	Non-Deterministic Performance	Disallowed: Wraps compiler <code>malloc()</code> , slow, thread-unsafe.
<code>heap_4.c</code>	Zero Fragmentation (Auto-Coalescing)	Optimal: Automatically merges freed RAM blocks.
<code>heap_5.c</code>	Fragment Proof across Multiple Regions	Overkill: Only needed if combining split RAM chunks.

Table 1: FreeRTOS Heap Allocation Comparative Performance Metrics

2 Project Workspace File Structure Directory Isolation

When you copy the complete ThirdParty middleware stack into your workspace root, the source tree contains alternative build architectures (IAR, Keil, RVDS) along with dozens of incompatible processor cores. Leaving these files inside your workspace will trigger compiler cross-definition failures.

You must clean up and prune your file explorer down to this exact directory layout:

```
1 [Your_Project_Root_Folder]
2     Core/
3         Inc/
4             FreeRTOSConfig.h      <-- Process
5 Configuration Matrix File
6         Src/
7             stm32u5xx_it.c        <-- System Interrupt
8 Target Routines
9     Third_Party/
10         FreeRTOS/
11             Source/
12                 include/          <-- Core Kernel API
13 Headers (.h files)
14                 tasks.c           <-- Task Management Core
15                 queue.c           <-- Inter-task Messaging
16 Engine
17                 list.c            <-- Scheduler Tracking
18 Infrastructure
19                 timers.c          <-- Software Timer Engine
20                 event_groups.c    <-- Multi-event
21 Synchronization
22                 portable/
23                     GCC/
24                         ARM_CM33_NTZ/ <-- DELETE ALL
25 OTHER COMPILER CORES
26                     port.c         <-- Core Task
27 Context Switching
28                     portmacro.h    <-- Core
29 Register Macros
30                     MemMang/
31                         heap_4.c     <-- DELETE heap_1,
32 2, 3, AND 5
```

Listing 1: Pruned and Validated Source Tree Topology

3 IDE Compiler Property Path Configurations

Because you are managing these source blocks manually rather than relying on automatic code generation tools, the integrated GCC compiler cannot automatically resolve nested header routes. If you attempt a build without adding these paths, the compilation will stop immediately with a fatal error: "FreeRTOS.h: No such file or directory".

Follow these steps to explicitly link the file paths inside **STM32CubeIDE**:

1. Right-click your root project folder in the left-hand **Project Explorer** window panel.
2. Select **Properties** at the very bottom of the contextual menu options.
3. Expand the **C/C++ Build** tree item on the left panel, then select **Settings**.
4. Open the **Tool Settings** tab configuration pane in the center.
5. Navigate to **MCU GCC Compiler** → **Include paths**.
6. Click the **Add... (Plus Icon)** tool button, select the **Workspace...** target, and map these folders:

```
1 "${workspace_loc}/${ProjName}/Third_Party/FreeRTOS/Source/include}"
2 "${workspace_loc}/${ProjName}/Third_Party/FreeRTOS/Source/portable/GCC/
   ARM_CM33_NTZ}"
```

Listing 2: Compiler Directory Include Paths to Paste

Pro-Tip: Using Dynamic Variables

By using the environment variable string `${ProjName}`, your path links remain fully portable. If you rename your project folder in the future, the compiler will update the references automatically without breaking your project paths.

4 Kernel Tuning Configurations (FreeRTOSConfig.h)

The `FreeRTOSConfig.h` header serves as the runtime configuration matrix for your operating system kernel. Copying this template from the `NUCLEO-U575ZI-Q` application folder ensures that your clock configurations and core register mappings match perfectly.

Verify that your config file matches these critical parameters to ensure system stability:

```
1 #ifndef FREERTOS_CONFIG_H
2 #define FREERTOS_CONFIG_H
3
4 /* System Architecture Tuning Parameters */
5 #define configENABLE_TRUSTZONE          0 /* Decouples secure memory
   requirements */
6 #define configRUN_FREE_RTOS_SECURE_ONLY 0 /* Grants OS full control
   over core */
7 #define configUSE_PREEMPTION            1 /* Enforces deterministic
   task switching */
8 #define configUSE_IDLE_HOOK             0
9 #define configUSE_TICK_HOOK             0
10
11 /* Hardware Clock Specification (Mapped to System Utility Registers) */
12 extern uint32_t SystemCoreClock;
13 #define configCPU_CLOCK_HZ              ( SystemCoreClock )
```

```

14 #define configTICK_RATE_HZ          ( ( TickType_t ) 1000 ) /* 1
    ms OS Tick Rate */
15
16 /* Memory Subsystem Resource Allocations */
17 #define configMAX_PRIORITIES        ( 56 )
18 #define configMINIMAL_STACK_SIZE    ( ( uint16_t ) 128 )
19
20 #define configTOTAL_HEAP_SIZE ( ( size_t ) ( 32 * 1024 ) ) /* 32KB
    Dynamic Pool */
21
22 /* API Service Inclusion Directives */
23 #define vTaskPrioritySet            1
24 #define uxTaskPriorityGet           1
25 #define vTaskDelete                 1
26 #define vTaskCleanUpResources      1
27 #define vTaskSuspend                1
28 #define vTaskDelayUntil             1
29 #define vTaskDelay                  1
30
31 #endif /* FREERTOS_CONFIG_H */

```

Listing 3: Validated FreeRTOSConfig.h Code Matrix for STM32U585

5 Hardware Interrupt Mapping & Low-Level Remapping

By default, the micro-controller relies on a bare-metal vector table initialized inside `stm32u5xx_it.c`. To run a multi-threaded application, the FreeRTOS scheduler must take control of the primary system interrupts.

5.1 The Critical Interrupt Trio

- **SVC_Handler (Supervisor Call):** Used to run the very first task thread during kernel startup.
- **PendSV_Handler (Pendable Service Call):** Handles low-priority execution switches to perform register context saves and restores during thread context changes.
- **SysTick_Handler (System Tick):** The hardware timer interrupt that drives the internal system tick counter.

Open `Core/Src/stm32u5xx_it.c` and rewrite the default handlers to match this layout exactly:

```
1  /* Insert these function prototypes at the top of your stm32u5xx_it.c  
   file */  
2  extern void vPortSVCHandler(void);  
3  extern void xPortPendSVHandler(void);  
4  extern void xPortSysTickHandler(void);  
5  
6  /**  
7   * @brief This function handles System service call via SWI instruction  
8   *  
9   * @note Redirected to the FreeRTOS port layer to start the scheduler.  
10  */  
11 void SVC_Handler(void)  
12 {  
13     vPortSVCHandler();  
14 }  
15  
16 /**  
17  * @brief This function handles Pendable request for system service.  
18  * @note Redirected to the kernel to perform task context switching.  
19  */  
20 void PendSV_Handler(void)  
21 {  
22     xPortPendSVHandler();  
23 }  
24  
25 /**  
26  * @brief This function handles System tick timer.  
27  * @note Services the HAL baseline clock and increments the OS kernel  
28  *       tick.  
29  */  
30 void SysTick_Handler(void)  
31 {  
32     HAL_IncTick();           /* Keeps standard STM32 HAL peripheral  
                               timeouts working */  
33     xPortSysTickHandler();   /* Increments the kernel tick counter */  
34 }
```

Listing 4: Vector Routine Modifications inside `stm32u5xx_it.c`

6 Main Thread Control Loop Implementation

Open your project's main application launcher file at Core/Src/main.c. Initialize your hardware peripherals, register your tasks, and pass system control to the scheduler loop.

```
1 #include "main.h"
2 #include "FreeRTOS.h"
3 #include "task.h"
4
5 /* Task Function Definition Prototype */
6 void Active_Heartbeat_Task(void *pvParameters);
7
8 int main(void)
9 {
10     /* Reset of all peripherals, Initializes the Flash interface and
11     SysTick. */
12     HAL_Init();
13
14     /* Configure the system clock to your desired target frequency */
15     SystemClock_Config();
16
17     /* Initialize any required onboard peripherals or GPIO pins here */
18
19     /* Instantiate your custom worker thread node */
20     BaseType_t task_creation_status;
21     task_creation_status = xTaskCreate(
22         Active_Heartbeat_Task, /* Function Pointer */
23         "Heartbeat_Node", /* Debug Identification Name */
24         128, /* Allocated Stack Depth (words) */
25         NULL, /* Task Input Parameter Pointer */
26         tskIDLE_PRIORITY + 1, /* Operational Execution Priority */
27         NULL /* Task Tracking Hook Reference */
28     );
29
30     if (task_creation_status == pdPASS)
31     {
32         /* Hand execution over to the scheduling engine loop */
33         vTaskStartScheduler();
34     }
35
36     /* Execution will never reach here unless the RAM heap is exhausted
37     */
38     while (1)
39     {
40         /* Trap execution if a fatal system error occurs during
41         allocation */
42         CRITICAL_ASSERT_FAULT();
43     }
44 }
45
46 /**
47  * @brief Application logic worker thread node.
48  * @param pvParameters: Input reference pointer arguments.
49  */
50 void Active_Heartbeat_Task(void *pvParameters)
51 {
52     while (1)
53     {
```

```
51      /* Insert your main application or peripheral control logic  
52      here */  
53      /* Block context cleanly for exactly 500 milliseconds */  
54      vTaskDelay(pdMS_TO_TICKS(500));  
55  }  
56 }
```

Listing 5: Production Application Setup Loop in main.c

7 Troubleshooting & Compilation Architecture Protocols

7.1 Resolving Code Cache Out-of-Sync Errors

Because you deleted several nested directories while editing the raw workspace, the eclipse indexing engine may look for files that no longer exist, causing “ghost” compilation errors.

To clear this file tracking cache:

1. Right-click your project name root node inside the **Project Explorer** toolbar.
2. Hover your mouse tracking over the **Index** options tab item.
3. Click on **Rebuild**. This will update the internal file index and clear any incorrect errors.

7.2 Common Build Errors Reference Table

If your project compilation flags any errors, reference this troubleshooting guide to identify the root cause:

Observed Console Error Output	Root Failure Cause	Corrective Resolution Action
Error: duplicate definitions of SVC_Handler	Standard template conflict in files.	Open <code>stm32u5xx_it.c</code> and delete or comment out the old bare-metal block.
undefined reference to <code>vApplicationGetIdleTaskMemory</code>	<code>configSUPPORT_STATIC_ALLOCATION</code> flag is active without an idle task hook.	Change the configuration flag back to 0 inside <code>FreeRTOSConfig.h</code> to use dynamic allocation.
OS Tick runs slow / Peripherals time out intermittently	Both HAL and FreeRTOS are competing for the SysTick timer resource.	Mandatory step: Change your project's HAL Timebase Source to a hardware timer like TIM6 or TIM7.

Table 2: FreeRTOS Workspace Integration Troubleshooting Metrics

7.3 The HAL Timebase Conflict Fix

By default, the STMicroelectronics HAL library and the FreeRTOS scheduler engine both try to utilize the hardware SysTick timer at the same time. This competition will distort standard delay calculations and cause peripheral tracking crashes.

To fix this conflict, move the HAL library timebase source to an alternate hardware peripheral timer:

1. Open your project configuration settings inside the left sidebar.
2. Navigate to **System Core** → **SYS**.
3. Locate the option property labeled **Timebase Source**.
4. Change the selection value from SysTick to any available basic hardware timer resource, such as TIM6 or TIM7.
5. Save your configuration and re-run your project compilation by clicking the Hammer Icon (Ctrl + B).